NAME OF STUDENT:	**STUDENT NUMBER:** .....

SUBJECT:	**RELEASE DATE:**	**DUE DATE:**	**TOTAL MARKS:**
ADP372S – APPLICATIONS DEVELOPMENT PRACTICE 3	09th June 2022	18th June 2022	100

Cape Peninsula University of Technology

creating futures

DEPARTMENT OF INFORMATION TECHNOLOGY
FACULTY OF INFORMATICS AND DESIGN

COURSE(S): DIPICTA: DIPLOMA IN INFORMATION AND COMMUNICATION TECHNOLOGY: APPLICATIONS DEVELOPMENT

EXAMINER: K NAIDOO
MODERATOR: R BURGER

SPECIAL INSTRUCTIONS

- Answer all the questions.

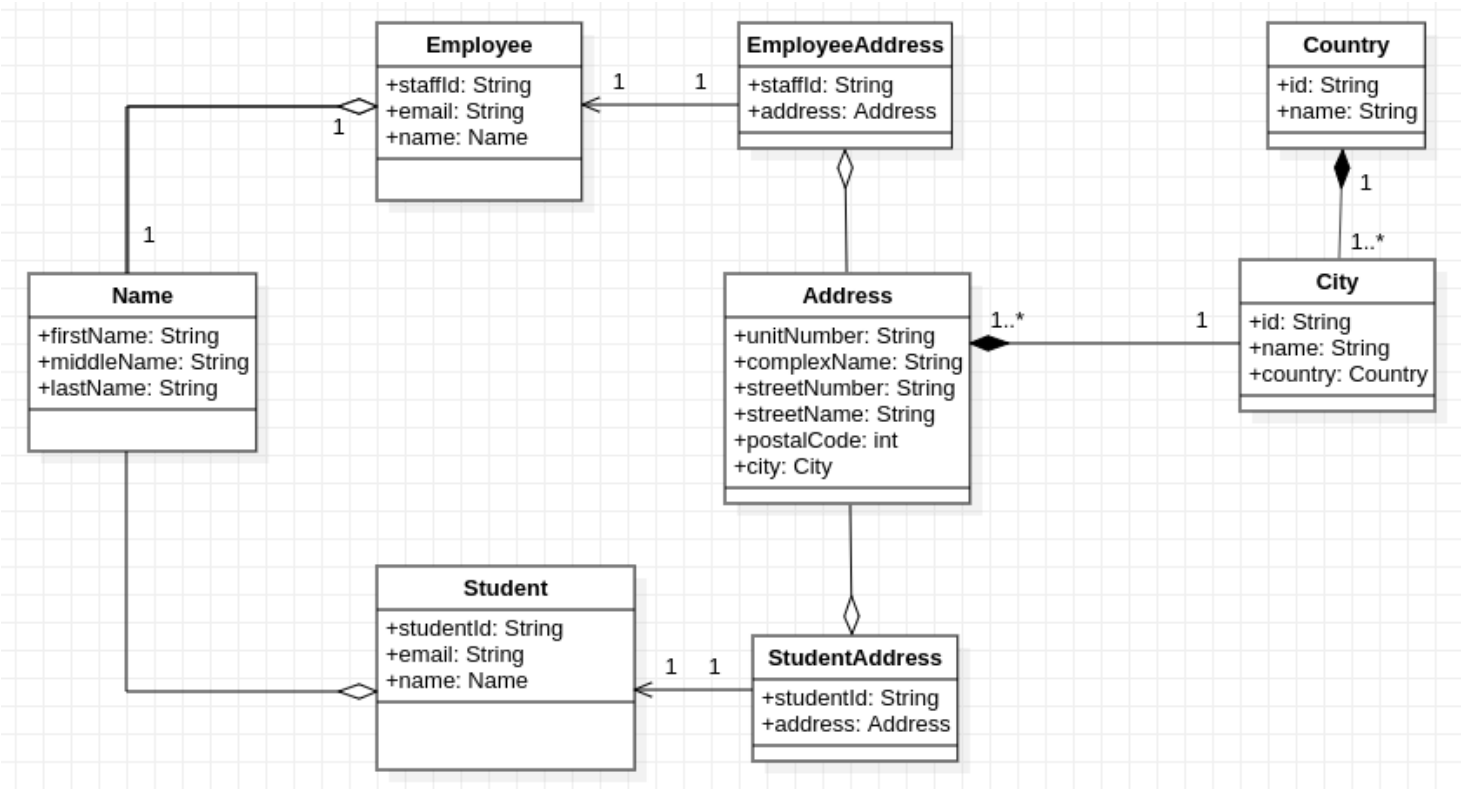
REQUIREMENTS

- IntelliJ IDEA

Problem Statement

In the quest for seamless and effective organisational processes in a college, the technology department of the college has been given the task to digitize and automate its academic-facing processes.

The technology department has decided to use Domain Driven Design to actualize this. A section of the technology team was given the responsibility to solicit requirements based on the as-is processes. Thus, after many analysis meetings, they have come up with a domain model. A section of the domain model is presented below:



The following **rules** hold true for the above domain model, and must be adhered to:

- All attributes of Employee, EmployeeAddress, Student, StudentAddress, Country and City are mandatory.
- Mandatory attributes of Address: streetNumber, StreetName, postalCode, city.
- Mandatory attributes of Name: firstName, lastName.
- Do not change the data type of any attributes in the domain model.
- Identifiers model the primary key of an entity. Identifiers for all entities in the domain model must be provided in your application and must NOT be generated by the system.
- All mandatory attributes must be validated before an instance of the corresponding entity is created. Throw an IllegalArgumentException for any validation violation.
- Attribute postalCode in Address must be a 4-digit number between 1000 and 9999.
- No attribute should be saved as null. Empty string is acceptable in place of null.
- Ensure that email addresses are valid.
- Both Name and Address are modelled as complex value objects.
- Identifier rules are as displayed in the table below:

Model	Identifier Attribute
Name	No identifier
Employee	staffId
Student	studentId
Address	No identifier
Country	id
City	id

- Use only Junit 5 as testing framework.

Your responsibility as part of the implementation team is to convert the above domain model into code written in Java, with your knowledge of Test Driven Development.

Instructions:

1. Create a Maven Java project called **school_management.**
2. In the root package of your project, create **domain**, **factory**, **repository** and **service** packages.
3. Use the appropriate design pattern where necessary.
4. Use helper classes and methods for re-occurring functionalities.
5. Consider the abstraction and adoption of good packaging principles in your project.
6. Attempt ALL questions. Your group should collaborate on **Github** and clearly show the work undertaken by all team members with issues, milestones and commits done timeously. Remember the team leader is the person responsible for setting up the base code and then all team members are to invited to clone the (master) base project to do their part. The team lead will only accept pull requests from the team members and merge if the code is error-free and can be integrated successfully into the project.
7. **Once your solution is completed, the group must ensure that ALL team members are in sync with the master branch of the team leader. Then each student must submit their Github link on Blackboard on/before the due date**.
8. There will **NOT** be any other form of submission except on Blackboard. Email submissions will **NOT** be considered.
9. **This is a group assessment and you have already been assigned to a formal group in ADP372S**. Only discuss this assessment with your team members and no one else. Any form of plagiarism may result in the group being severely penalised**.**
10. Students may be penalised for not adequately contributing in the project code.

Questions:

Q1. In the **domain** package, implement models for all the entities and value object in the domain model. Use the Builder pattern and adhere to the rules.

Q2. Implement

a) Factories for the entities and value object in the **factory** package.

b) Test cases for the factories.

Q3. Using JPA (Java/Jakarta Persistence API), now implement the repository layer in the **repository** package. Your repository implementation may include the additional operations required by the services in your application.

Q4. In the **service** package, implement services to consume your repository implementation with similar operations offered by JPA *where appropriate*:

a) Save operation.

i. Implementation

   ii. Test case

b) Read one operation.
   i. Implementation
   ii. Test case

c) Delete operation which returns nothing.
   i. Implementation
   ii. Test case

d) Read all operation.
   i. Implementation
   ii. Test case

e) Use of generic interface

f) Use of domain-specific interface

Q5. Code a service to get the employee name given an employee email. Check if the email is valid and exists.

Q6. Implement a service to retrieve all employee name(s) living in a particular city. The formal parameter passed is the cityId.

Q7. Code a service to return the list of all cities in a given country. The parameter passed is the countryId. The return value is either a sorted list of city names or null (if no cities found for that countryId).

Q8. Using your knowledge of Facade design pattern, implement a service to retrieve a list containing the last name of all the students in the same country, given a country id. Hint: Revisit Q6 to ensure that it was implemented correctly.

Q9. Now, expose the services created in Q5, Q6, Q7 and Q8 (above) as webservices in the controller layer of the **controller** package. Write appropriate test cases for these implementations.

Q10. Finally, discuss with your team the improvements that you could make to this solution, and possibly some obvious flaws in both the (uml domain) design and implementation of the system. Note these in the “Readme.md” of your project.

Happy coding!

Special Conditions to take note of:

1. Student(s) not participating in the group assessment will be awarded 0%.
2. Student(s) not contributing adequately to the group assessment may be penalised.
3. Evidence of group work and collaboration that is evenly distributed must be clearly shown on Github.com so the team members are awarded the same mark. Ensure that your group sets milestones and issues(tasks). The team lead should only merge error-free pull requests. Team members may be requested to answer questions posed about their contribution before marks for the assessment are finalised.
4. Any form of plagiarism will be dealt with severely and the usual University processes will be adhered to in this regard, so please refrain from discussing this assessment outside the members of your group. Student(s) committing any form of plagiarism may be awarded 0%.